SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I _G. Srivalli (192365014)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Srivalli

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Question 1: HTTP Response Analysis

Provided Data

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Analytical Breakdown

1. Identification of Incorrect/Invalid Header Fields

- **Invalid `Content-Length` Value:** Under RFC 7230 and RFC 9110, the `Content-Length` header field must consist of a non-negative decimal integer representing the exact size of the payload body in octets (bytes). The value `-150` is syntactically invalid.
- **Header Line Delimitation:** Standard HTTP protocol transmission requires each header line to be explicitly terminated by a Carriage Return and Line Feed (`CRLF`, or `\r\n`) sequence. Merging header lines or delivering incorrect line breaks prevents standard HTTP parsers from tokenizing headers correctly.

2. Technical Impact of `Content-Length` Misconfiguration

- **Parser Exceptions & Protocol Violations:** Modern web browsers and HTTP client libraries enforce strict structural validation. Encountering a negative payload length triggers immediate header parsing exceptions, leading to terminated connections or thrown client-side network errors.
- **Stream Desynchronization & Request Smuggling:** When `Connection: keep-alive` is designated, the persistent TCP socket connection relies on precise frame boundaries demarcated by `Content-Length` (or `Transfer-`

`Encoding: chunked`). A negative byte length corrupts the parser's payload offset calculations. Proxies and downstream servers lose track of request boundaries, creating vulnerabilities to HTTP Request Smuggling or causing catastrophic connection stalls.

3. Alignment of Status Code with Payload Integrity

The status code `200 OK` indicates successful processing at the application layer. However, the transport framing is fundamentally broken. If an application logic fault caused a failed payload size calculation, the server should abort payload delivery and emit a `500 Internal Server Error` instead of emitting corrupted framing headers alongside a successful status indicator.

4. Remediation & Standardized Output

Supply a valid, non-negative byte count corresponding strictly to the outgoing payload body (or transition to dynamic chunked streaming).

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
[150 bytes of HTML content]

Question 2: DOM Structure Analysis

Provided Code

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Analytical Breakdown

1. Identification of Structural Violations

- The paragraph element `<p>` is instantiated prior to the content string `"HelloWorld"`, but lacks a corresponding explicit closing tag `</p>` prior to the closure of its parent container `</div>`.

2. Browser Parser Interpretation Mechanics

- Modern browser engines follow the HTML5 parsing specification, which mandates auto-correction and error-recovery routines.
- Upon encountering the closing tag `</div>` while an open `<p>` context exists, the parser automatically injects an implied closing tag `</p>` to clear the element stack before closing the parent block.

DOM Tree Construction Structure:

```
html
 └── head
 |    └── title ("TestPage")
 └── body
      └── div
           └── p ("HelloWorld")
```

3. Impact on Rendering, Layout, and Compliance

- **Standard Visual Rendering:** Modern layout engines will render "HelloWorld" inside a block-level paragraph within the parent container without visible breakdown.
- **Styling & Scripting Anomalies:** Structural auto-correction can cause unexpected results when targeting nodes with descendant CSS selectors (`div > p`) or executing structural DOM traversal via JavaScript.
- **Strict Context Execution:** Under strict XHTML or XML parsers, unclosed elements violate well-formedness rules, causing fatal parsing exceptions that halt document processing entirely.

4. Validated Standard-Compliant Structure

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

Question 3: GET vs POST in Login Systems

Provided Data

Parameter / Metric	HTTP GET Method	HTTP POST Method
Endpoint Format	`/login?user=admin&pass=1234`	`/login`
Data Visibility	Fully exposed in URI query string	Contained within Request Payload Body
Security Profile	High Exposure / Low Security	Protected Payload / Moderate-High Security

Analytical Breakdown

1. Fundamental Transmission Differences

- **Payload Location:** `GET` serializes parameters directly into the Request-URI. `POST` packages parameters within the HTTP request entity body, completely separating input data from the target path.
- **Semantic Purpose:** `GET` is defined by RFC 9110 as a safe, idempotent method intended strictly for resource retrieval. `POST` is designed for state-changing, non-idempotent processing tasks.
- **Length Constraints:** `GET` requests are bound by browser and proxy URI length limits (typically ~2,048 characters). `POST` requests have no standardized size limit, enabling large payloads.

2. Method Selection Analysis for Authentication

`POST` is the required method for handling authentication workflows:

- **Credential Masking:** Credentials do not appear directly within the browser address bar or user interface.
- **Persistence Prevention:** Prevents sensitive data from being cached in browser history, saved in bookmarks, or stored in plaintext web server access logs.
- **Cryptographic Layering:** Works with TLS encryption (HTTPS) to secure headers and request body content during transit.

3. Security Vulnerabilities Associated with GET Authentication

- **Web Server Access Logging:** In standard infrastructure, web servers (e.g., Nginx, Apache) record incoming Request-URIs in plaintext log files (`access.log`), exposing plain text user passwords.
- **Referer Header Information Leakage:** Navigating to an external link from an authenticated `GET` endpoint transmits the full URI—including embedded parameters—to external hosts via the `Referer` header.
- **Browser Caching & History Storage:** Input query strings persist indefinitely in client-side history, session restoration buffers, and intermediary caching proxies.

4. Best Practices for Production Authentication

- Mandate Transport Layer Security (**HTTPS / TLS 1.3**) across all endpoints.
- Transmit credentials using `POST` containing JSON or URL-encoded form data in the request body.
- Implement **Anti-CSRF tokens** or apply strict `SameSite` attribute controls on session cookies.
- Issue session IDs using secure cookie flags: `HttpOnly`, `Secure`, and `SameSite=Strict`.
- Enforce server-side rate limiting and brute-force mitigation mechanisms.

Question 4: HTML Code Inspection

Provided Code

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome<p>This is a paragraph
<img src="image.jpg">
</body>
</html>

Analytical Breakdown

1. Identification of Omitted Tags

- **Unclosed Heading Tag:** The primary heading `<h1>Welcome` lacks a closing `</h1>` tag prior to the paragraph declaration.
- **Unclosed Paragraph Tag:** The paragraph element `<p>This is a paragraph` lacks a closing `</p>` tag prior to the image tag and body closure.

2. Accessibility and Layout Issues in the `<img>` Element

- **Missing `alt` Attribute:** Violates Web Content Accessibility Guidelines (WCAG). Screen readers cannot interpret or read out the image contents to visually impaired users, and alternative text cannot be shown if image loading fails.
- **Omitted Explicit Dimensions:** Lacking explicit `width` and `height` attributes prevents the browser from calculating layout space during DOM construction, causing Cumulative Layout Shift (CLS) as assets load.

3. Browser Parsing Logic and Execution

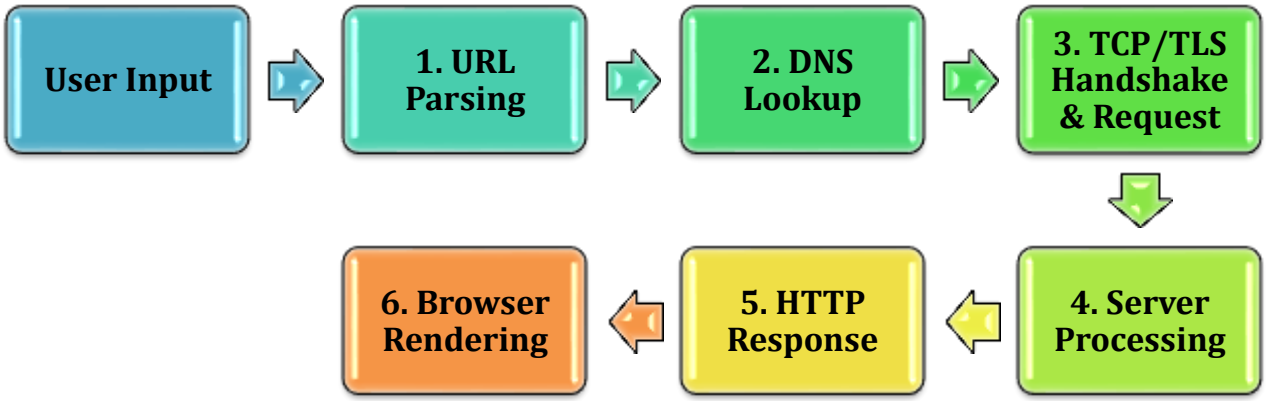
- **Implicit Tree Reconstruction:** Modern HTML5 parsers automatically insert closing tags when encountering new block elements. Encountering `<p>` forces the auto-closure of `<h1>`; encountering `</body>` forces the auto-closure of `<p>`.
- **Side Effects:** While content displays sequentially, missing attributes lead to lower search engine optimization (SEO) scores, reduced accessibility compliance, and visual layout shifts.

4. Fully Corrected Standard-Compliant HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sample</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph</p>
    <img src="image.jpg" alt="Sample descriptive image" width="400" height="300">
</body>
</html>
```

Question 5: Browser-Server Communication Flow

Overview of Communication Sequence



Analytical Breakdown

1. Step-by-Step Technical Execution

- **Step 1: URL Entry & Parsing:** The browser breaks down the user input into its components: protocol (`http`/`https`), host domain (`example.com`), optional port, and resource path (`/index.html`).
- **Step 2: Domain Name Resolution (DNS):** The client resolves the domain name to an IP address by checking browser caches, OS host files, resolver caches, and running recursive DNS queries.
- **Step 3: Protocol Connection & Request Transmission:** The client performs a TCP 3-way handshake (plus a TLS handshake for HTTPS) to establish a connection, then transmits the HTTP request containing request headers and optional body content.
- **Step 4: Server-Side Execution:** The web server receives the request, parses headers, executes backend scripts, queries connected databases, and generates an HTTP response.

- **Step 5: Response Delivery:** The server returns the payload accompanied by an HTTP status code, response headers, and encoded entity data.
- **Step 6: Client Rendering Engine Pipeline:** The browser parses the HTML markup to build the Document Object Model (DOM), parses CSS to build the CSSOM, combines them into a Render Tree, calculates layout geometry (reflow), and paints visual elements to the screen.

2. Latency and Performance Bottlenecks

- **DNS Resolution Delays:** Uncached domain lookups require sequential queries across root, TLD, and authoritative name servers, adding network delay.
- **Transport Level Latency (RTT):** Establishing TCP and TLS connections across multiple network round-trips adds connection overhead before data transmission begins.
- **Server Time-To-First-Byte (TTFB):** Unoptimized database queries, slow backend operations, or un-cached dynamic page rendering delay the initial server response.
- **Critical Rendering Path Blockers:** Synchronous, non-deferred JavaScript files and render-blocking external CSS stylesheets delay page layout execution.

3. Core Network Functions of DNS

- Serves as a distributed directory mapping user-friendly hostnames to IP addresses.
- Acts as the primary entry point for web communication; unresolved DNS queries prevent network packet delivery.
- Enables load distribution across infrastructure using GeoDNS routing, round-robin setups, and Content Delivery Network (CDN) edge routing.

4. Architecture Optimization Strategies

Layer	Optimization Strategy	Implementation Mechanism
DNS / Sockets	Connection Pre-allocation	Add `<link rel="dns-prefetch">` and `<link rel="preconnect">` tags.
Transport	Protocol Upgrade & Edge Routing	Deploy HTTP/2 or HTTP/3 (QUIC) over an Anycast Content Delivery Network (CDN).
Payload	Compression & Cache Rules	Apply Brotli/Gzip compression and configure explicit `Cache-Control` headers.
Rendering	Non-blocking Pipeline	Attribute non-critical scripts with `async` or `defer`, and inline critical CSS.